

# Lab 0: React Native and Expo Setup Guide

This guide is intended to guide students with installing React Native and then Expo, a React Native framework that allows developers to build apps across Android and iOS with JavaScript with a focus on faster iteration cycles.

**This lab assumes you have a Mac OS operating system. Slight modifications if you are running Windows, but the basic ideas are the same.**

## Part 1: Installing Dependencies

Before we start, we need to install some dependencies! The following instructions are adapted from the official React Native documentation found [here](#).

Let's install some dependences. Install Node and Watchman using the terminal: `brew install node brew install watchman` Note: If you have node already installed, make sure to upgrade your node version to the latest one or have a version that is 18.18 or newer. You can do this by running `node -v`. If you need to upgrade, you can run `npm i -g n`.

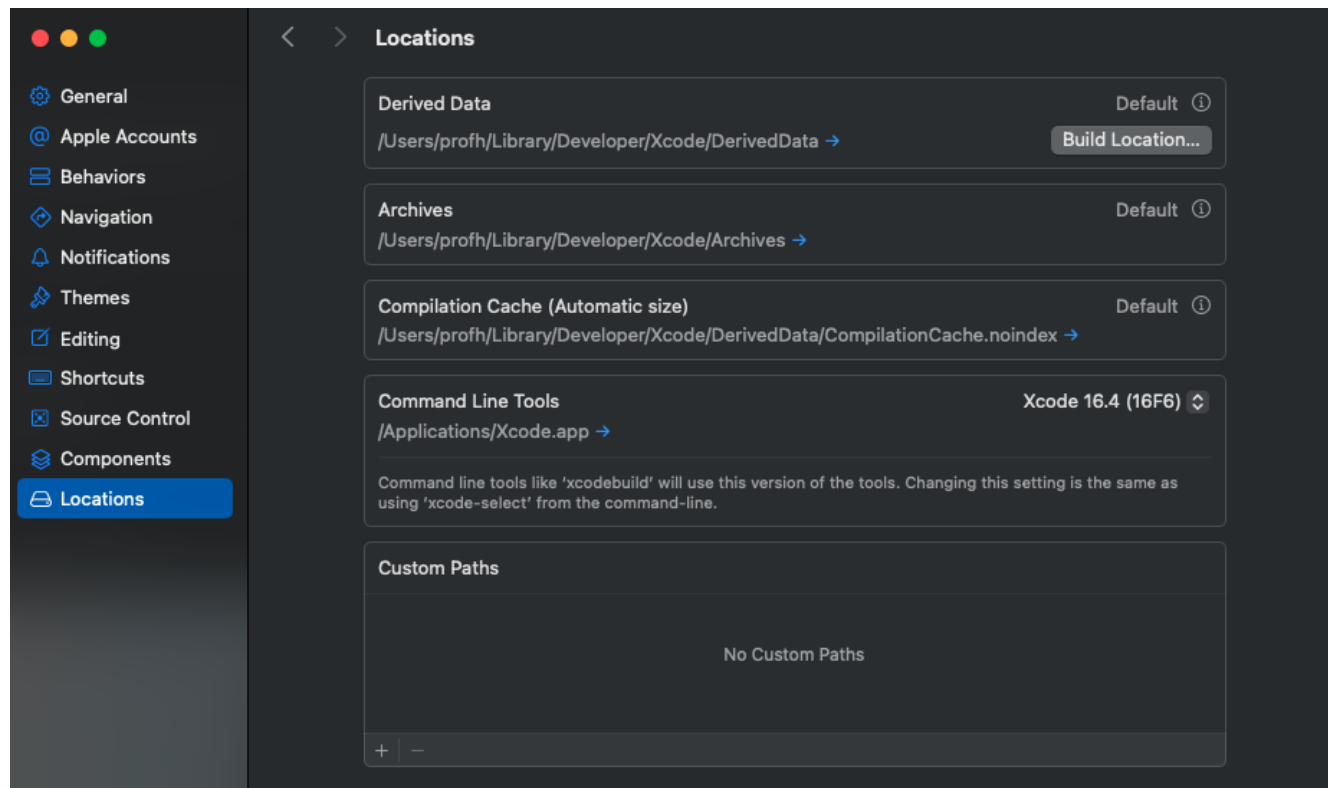
## Part 2: Installing XCode and Android Studio

Now, let's install XCode and Android Studio. These programs have iOS and Android emulators that allow us to build our app to either platform and test them out!

### XCode Installation

We can go to the App Store to download Xcode! Make sure to also down the iOS emulator when prompted! You can do this by going to "Xcode > Settings > Platforms", and then click "+" icon to select an iOS version.

After it is installed, we also need to install Xcode Command Line Tools. Click on "Xcode" on the top left, then "Settings". Click on "Locations" on the far right. The UI should look something like this:



Lastly, install the Cocoapods gem by running `sudo gem install cocoapods` in the terminal. Cocoapods is a system for handling dependencies for iOS; we will mostly use Swift Package Manager for this, but React Native relies on Cocoapods (which is fine).

Make sure to install a device to emulate in Xcode. You can do this by selecting "Product > Destination" and selecting a device to emulate on (we recommend the iPhone 16 for development).

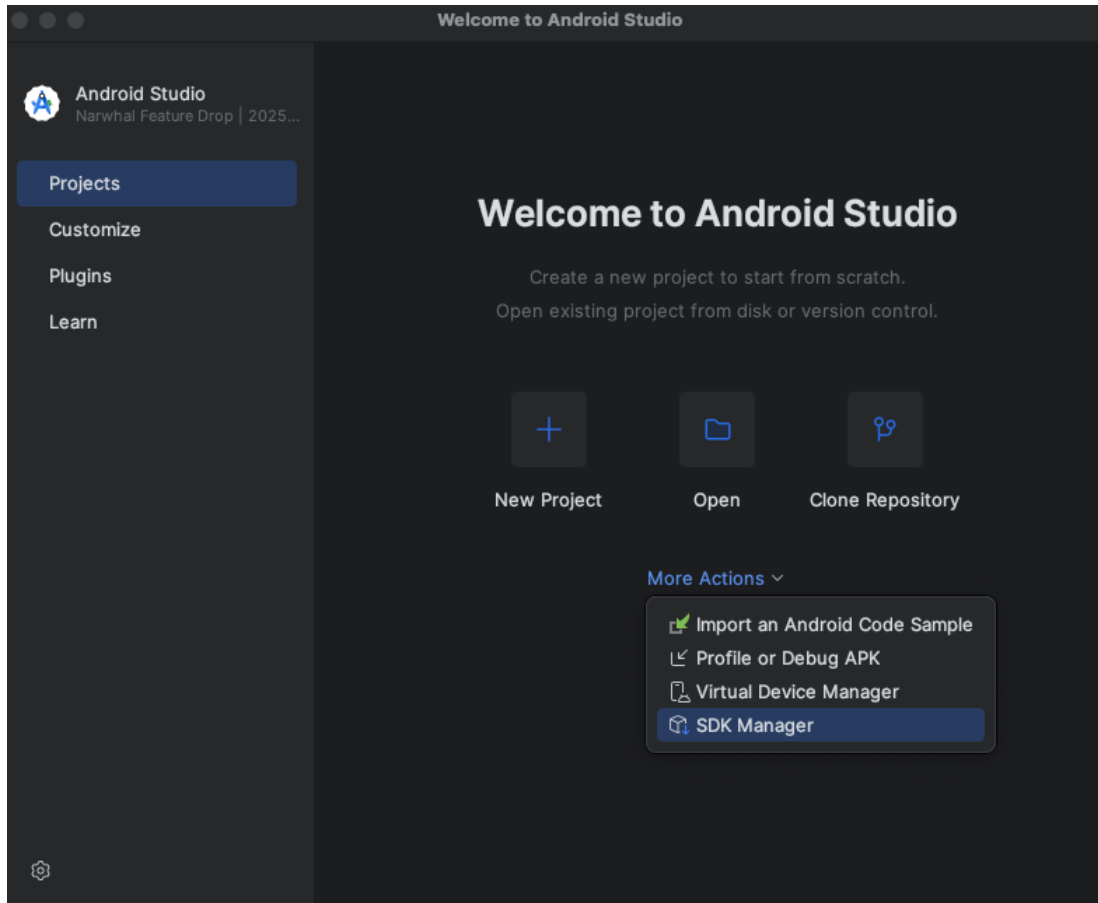
## Android Studio Installation

**IMPORTANT:** You only need to download Android Studio if you want to test your apps out on an Android device! If you plan on only doing iOS development with React Native, you can skip this step!

Download Android Studio [here](#). Once the dmg file is downloaded, drag it to applications and then double click it (this will open the installation wizard). In the installation wizard, make sure Android SDK, Android SDK Platform, and Android Virtual Device are installed.

Note: If you select the "Standard" procedure during installation, Android Studio will install almost all the necessary dependencies that you will need!

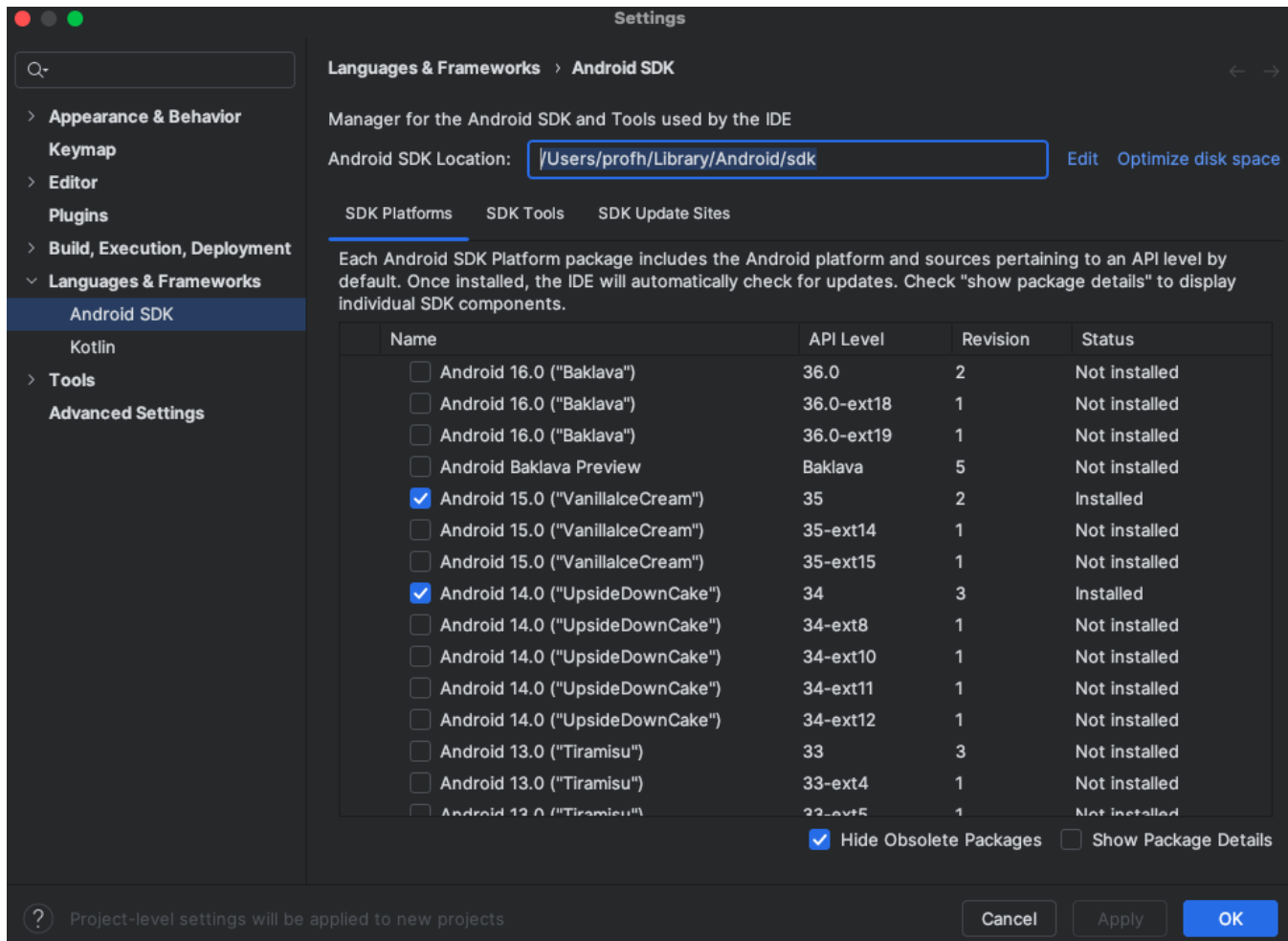
Once Android Studio has been installed, the UI will look something like this:



In Android Studio, click "More Actions" on the landing page and then "SDK Manager" to install a few extra things, as seen in the screenshot above.

Under "SDK Platforms", install/verify the latest Android stable version (as of now it is Android 14.0 ("UpsideDownCake")) as well as the latest version of the Android API (as of now it is Android API 35).

Under "SDK Tools", install/verify Android SDK Build-Tools 35, Android Emulator, and Android SDK Platform-Tools. The window should look like this:



Before you close the window, copy the "Android SDK Location" path at the top!

Finally, we need to add the Android SDK location and platform tools as an environment variable to our path. To open up our zsh profile (bash also works!), run `open ~/.zshrc` in the terminal and copy and paste the following in at the bottom of your zsh profile:

```
export ANDROID_HOME=<sdk path you copied> export PATH=$PATH:$ANDROID_HOME/emulator export PATH=$PATH:$ANDROID_HOME/platform-tools
```

Save this file and in the terminal run `source ~/.zshrc` to make sure your changes take effect!

### Part 3: Verifying your Android Studio setup

At this point, run `adb` in your terminal. If you get a "command not found", reread the above steps to make sure you have properly installed everything! If the command works, then we can setup our Android Virtual Device which will be our Android emulator.

On the welcome page, select "Virtual Device Manager". When creating a new phone device, we recommend the latest version of the Pixel that has Play Store already installed. Next, we need the operating system for our Android emulator. Select "Android API 35" in the Recommended tab.

Finally, make a name for your virtual device and click "Finish".

Congrats, you now have successfully setup your iOS and Andorid emulators!

### Part 3: Installing Expo

Run `sudo npm i -g expo-cli` in you terminal to install the Expo CLI. Expo CLI will allow us to start an Expo server and deploy our app extremely fast to our devices. Note: You might get some warnings during the installation process but you can ignore them!

Also, on your *mobile device*, install "Expo Go" on the App Store or Google Play Store. This would allow Expo to run our React Native Code that is on our laptop, bundle it, and send it to your phone!

### Part 4: Setting up VSCode

You might be familiar with VSCode which is what we will use to do all of our code editing when working with React Native. We highly recommend installing the following extensions:

- React Native Tools by Microsoft (allows us to debug React Native code within our IDE)
- React Native/React/Redux snippets by Equimper (enhances autocompletion of React Native Components)
- Prettier Code Formatter by Esben Peterson (formats our code automatically)
- Material Icon Theme (applies icons to our files for better identification)

## Part 5: Creating an Expo Project

In a new folder, run the command to create a new project! `npx create-expo-app --template`

Choose the *Blank (Typescript)* template and enter "Lab-0" as the app name. This will run `npm install` which will download a bunch of files! After waiting a bit, the starter code will be generated for you! You will know if it worked if you see the following in the terminal:

Don't actually follow the instructions below! We will run our project using Expo's managed project workflow (more on this later)!

✓ Your project is ready!

To run your project, navigate to the directory and run one of the following npm commands.

```
- cd <project name>
- npm run android
- npm run ios
- npm run web
```

Navigate into the generated folder and open up App.tsx. This is the "starting point" for our project.

The starter code code in App.tsx will look like this:

```
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text>Open up App.js to start working on your app!</Text>
      <StatusBar style="auto" />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

## Part 6: Deploy the Expo App

To run and deploy this project, switch to the Lab-0 directory and run `npx expo start` in the terminal.

You should see the following popup:

```
› Metro waiting on exp://192.168.12.105:8081
› Scan the QR code above with Expo Go (Android) or the Camera app (iOS)

› Using Expo Go
› Press s | switch to development build

› Press a | open Android
› Press i | open iOS emulator
› Press w | open web

› Press j | open debugger
› Press r | reload app
› Press m | toggle menu
› Press o | open project code in your editor

› Press ? | show all commands
```

Scan the QR code and it open the app on your device.

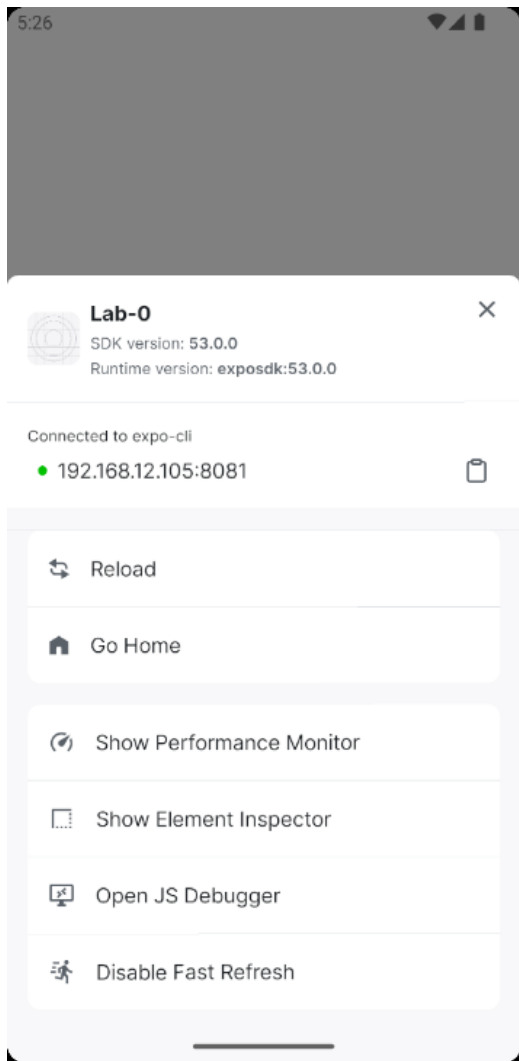
To deploy the app to your iOS and Android emulators, press "i" and "a" respectively. This will attach your app to those emulators on your laptop!

Note: If you get an error like `Error: EMFILE: too many open files`, delete the `node_modules` folder in the project and run `npm install` to reinstall it.

Note: If you get deprecation warnings like "punycode module is deprecated", you can ignore the warnings. They will not affect anything.

## Part 7: Accessing the Developer Menus

When developing, it is helpful to access to the developer menus that Expo has. If you press "Cmd-D" and "Cmd-M" for iOS and Android emulators respectively, the developer menu will popup and should look something like this (this is the Android emulator):



(If "Cmd-D" doesn't work for iOS simulators, try "Command-Option-Z").

The developer menu has handy features like the JS debugger to debug Javascript code as well as monitoring on the Console and Network (just like how Chrome's WebTools work)! The one we will use most often is the "Reload" feature. React Native has a "fast refresh" feature where once you save your code changes, those changes are automatically reflected on the app. In the case where fast refresh does not work, we can force a reload of the app using the developer menu.

Note: Android emulators seem to not run well on macOS especially when trying to bring up the developer menu. If "Cmd-M" does not work, close the emulator and run `npx expo start` to reattach your code to the emulator.

Finally, if you haven't done so already, take out your phone and open up the camera app. Having run `npx expo start` in your terminal, a QR Code is generated that you can scan. This will redirect you to Expo Go which will run and then deploy your React Native code, allowing you to interact with your app on your phone!

Note: On a phone, open the developer menu by shaking the phone (you might have to shake it hard)! Deployments to your phone require that your laptop and your phone be on the same Wifi network (make sure to click "Allow" on all prompts in the Expo Go app)!

If this is working, you've installed everything you need to start React Native development in this class. Qapla'